When Does Graph Structure Help in Multi-Agent Reinforcement Learning? A Controlled Empirical Study

Anonymous authors
Paper under double-blind review

Abstract

Graph neural network (GNN) value-factorisation methods are widely claimed to outperform graph-free baselines in cooperative multi-agent reinforcement learning (MARL) by exploiting relational inductive bias. The conditions under which the claim holds are undercharacterised: prior work typically reports a single benchmark in a single graph regime, conflating the contribution of the graph with that of added capacity, and leaving the alignment between the GNN's receptive field and the task's coordination structure unexamined. We close this gap with a controlled empirical study on CoordGrid, a cooperative grid-world whose coordination graph is itself the experimental variable. Holding the per-agent encoder, state-conditioned mixer, optimiser, and exploration schedule fixed across four algorithms (IQL, VDN, QMIX, GNN-QMIX), we test three pre-registered hypotheses — on a structured graph (H1), against a random graph of matched edge density (H2), and across a GNN-depth × graph-diameter grid (H3). Three findings emerge. First, on every (N, graph) cell we tested ($N \in \{4, 8\} \times \{\text{ring, Erdős–Rényi at matched edge density}\}$), GNN-QMIX — a deliberately minimal GCN-augmented QMIX— is worse than structurally identical QMIX (one-sided Welch t, H_a: GNN-QMIX > QMIX, $p \geq 0.98$; no evidence in favour of the graph-augmented variant), with gaps ranging from -60 to -147 return units; undertraining of the larger model remains the leading alternative explanation and is discussed explicitly. Second, H2's directional prediction (structure helps GNN-QMIX more than matched-density random graphs) is supported at $N = 4$ ($p = 0.007$) but reverses at $N = 8$. Third, the receptive-field alignment heuristic ($L \approx d$) is falsified at the family level: the data reveal an abrupt over-depth cliff at $L \geq 3$, suggesting the practitioner rule "do not stack more than two GCN layers in front of a QMIX-style mixer in small-team CTDE settings." All claims are scoped to one GNN family (vanilla GCN) and to the CoordGrid environment. We release the harness, configs, and plotting code at the URL in Appendix C.

1 Introduction

Cooperative multi-agent reinforcement learning (MARL) underlies progress on team coordination problems from traffic signal control to multi-robot manipulation. Value-decomposition methods — in which a joint team Q-value Q_{tot} is factorised into per-agent utilities Q_i for centralised training and decentralised execution — have emerged as a workhorse: VDN (Sunehag et al., 2018) factorises Q_{tot} as a sum, QMIX(Rashid et al., 2018; 2020) as a monotonic mixture, and several recent methods extend the mixer with graph neural networks (GNNs) over an externally specified or learned coordination graph (Jiang et al., 2020; Liu et al., 2020; Böhmer et al., 2020; Naderializadeh et al., 2020).

The intuition for graph-based MARL is straightforward: when agents form a coordination *structure*, a model whose architecture mirrors that structure should generalise better and converge faster. This intuition is widely cited as motivation in the literature, yet the conditions under which it actually holds are surprisingly under-tested. Three specific gaps motivate the present study:

1. **Graph regime is rarely varied.** Most empirical comparisons fix one environment with one (often implicit) coordination graph, leaving open whether observed gains transfer to other graphs — sparse vs. dense, structured vs. random.
2. **Capacity is rarely controlled.** Adding a GNN adds parameters and depth as well as relational structure. Without a controlled comparison it is unclear whether the win comes from the graph or from the extra capacity.
3. **Receptive-field alignment is rarely studied.** A GNN of depth L propagates information across paths of length up to L. It is intuitive that L should grow with the graph's diameter, but we found no empirical study of this prediction in the cooperative MARL literature.

Contributions. This paper makes the following contributions.

1. We introduce CoordGrid, a cooperative gridworld whose coordination graph is a tunable knob (ring, line, complete graph, lattice, Erdős–Rényi). The reward is defined edge-wise over this graph, so the graph is literally the coordination structure rather than a heuristic over a fixed task.
2. We implement IQL, VDN, QMIX, and GNN-QMIX in a single harness with shared encoder, mixer, optimiser, and exploration schedule, so the only architectural difference between GNN-QMIX and QMIX is the GNN.
3. We pre-register three falsifiable hypotheses (Section 4) and test them with a pilot, a graph-regime sweep, and a depth × diameter ablation (Sections 6.1–6.4).
4. We find that on the pilot ring-graph condition (§6.1), the prevailing positive picture of graph-based MARL does not survive a controlled comparison: GNN-QMIX is bested by QMIX in both mean return and seed-level variance, despite the edge-defined reward that should maximally reward a graph-aware model. The depth–diameter alignment finding from H3 (§6.4) is the most actionable practitioner result.

2 Background and Notation

Decentralised partially observable Markov decision process (Dec-POMDP). A cooperative MARL task is a tuple $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, P, R, \{\mathcal{O}_i\}, \{O_i\}, \gamma \rangle$, where $\mathcal{N} = \{1, \ldots, N\}$ are agents, $\mathcal{S}$ the global state, $\mathcal{A}_i$ the discrete action set of agent i, $P(s' \mid s, \mathbf{a})$ the transition kernel, $R(s, \mathbf{a})$ a shared scalar reward, and $O_i(s) \in \mathcal{O}_i$ the local observation function. Agents share the discount γ and select actions from $\mathcal{O}_i$; centralised training (with access to s) is permitted while execution is decentralised (Oliehoek and Amato, 2016).

Centralised training with decentralised execution (CTDE) and value factorisation. Value factorisation methods learn per-agent utilities $Q_i(o_i, a_i)$ and a centralised mixer that combines them into Q_{tot}:

$$Q_{\text{tot}}(s, \mathbf{a}) = f_{\text{mix}}\big(Q_i(o_i, a_i)\,;\, s\big),$$

trained against a Bellman target on the team reward. The decentralised greedy policy $\pi_i(o_i) = \arg\max_{a_i} Q_i(o_i, a_i)$ recovers the joint greedy action when f_{mix} satisfies the Individual-Global-Max (IGM) condition (Son et al., 2019).

Coordination graph. We define a coordination graph $\mathcal{G} = (\mathcal{N}, \mathcal{E})$ over the agents. CoordGrid (Section 5.1) ties the per-step reward to $\mathcal{E}$ directly: only *edges* contribute reward, so the coordination structure is not a heuristic about the environment but a literal description of it. GNN-QMIX takes $\mathcal{G}$ as side information; IQL/VDN/QMIX do not.

3 Related Work

Value-decomposition MARL. IQL (Tan, 1993) trains independent Q-learners on the shared reward; biased, but the standard no-coordination baseline. VDN (Sunehag et al., 2018) sums per-agent utilities;

QMIX (Rashid et al., 2018; 2020) learns a monotonic state-conditioned mixer via a hypernetwork, enforcing $\partial Q_{\text{tot}}/\partial Q_i \geq 0$ so decentralised greedy execution is consistent with the centralised argmax. QTRAN (Son et al., 2019), MAVEN (Mahajan et al., 2019), and QPLEX (Wang et al., 2021) relax or extend the monotonicity assumption. Beyond value factorisation, MAPPO (Yu et al., 2022) is a strong actor-critic baseline; we restrict scope to value-factorisation methods to keep the architectural contrast with GNN-QMIX clean. RIAL/DIAL (Foerster et al., 2016) and CommNet (Sukhbaatar et al., 2016) *learn* an inter-agent communication graph and are out of scope here, where the graph is given.

Graph-based MARL. DGN (Jiang et al., 2020) applies graph convolution over a k-NN agent graph; G2ANet (Liu et al., 2020) uses attention to abstract a game graph; Deep Coordination Graphs (Böhmer et al., 2020) factor Q_{tot} over a hyperedge structure; GraphMIX (Naderializadeh et al., 2020) interleaves a GNN with a QMIX-style monotonic mixer (closest in spirit to our GNN-QMIX). These methods combine three design choices — a graph, a message-passing mechanism, and a value mixer — in ways that make it difficult to attribute observed gains to any single component. Our study isolates the GNN encoder by inserting it into an otherwise- identical QMIX.

Foundational GNNs and evaluation methodology. We use a textbook graph convolutional network (Kipf and Welling, 2017); Veličković et al. (2018), Xu et al. (2019), and Battaglia et al. (2018) characterise alternatives. The evaluation protocol follows Henderson et al. (2018) and Agarwal et al. (2021): multiple seeds, confidence intervals, explicit sample-efficiency metric, Holm-corrected pairwise tests, and effect sizes alongside p-values.

Benchmarks. The Multi-agent Particle Environment (Lowe et al., 2017) and PettingZoo (Terry et al., 2021) provide canonical cooperative tasks; SMAC (Samvelyan et al., 2019) is a heavier StarCraft micromanagement benchmark. We use CoordGrid as the primary test bed because it lets us *set* the coordination graph rather than infer it, and report a restricted MPE replication in Section 6.3.

4 Hypotheses

We test three pre-registered hypotheses:

H1 (structure beats no-structure):
On a cooperative task whose reward is graph-edge-defined, GNN-QMIXachieves a higher final return and crosses a fixed performance threshold sooner than IQL, VDN, and QMIX when the underlying coordination graph is sparse and structured.

H2 (structure > matched-density random):
The advantage of GNN-QMIX over QMIX is larger on a structured graph (ring) than on an Erdős–Rényi graph of the same edge density. This rules out the explanation that GNN-QMIX wins simply by having more inputs.

H3 (depth–diameter inverted-U):
Final return as a function of GNN depth L peaks near $L \approx d$, where d is the graph diameter, and degrades when L is either substantially smaller (insufficient receptive field) or substantially larger (over-squashing, optimisation noise). *Falsifiability criterion (pre-registered):* for each diameter d tested, we declare a per-row match if (a) the mean depth-return curve is non-flat (relative range $(\max - \min)/\max \geq 0.10$), and (b) $|L^* - d| \leq 1$ where L^* is the empirical argmax depth. We declare the family-level H3 confirmed if at least $\lceil 2n/3 \rceil$ of the n tested diameters pass the per-row criterion.

5 Experimental Design

5.1 The CoordGrid environment

CoordGrid is a cooperative gridworld parameterised by the team size N, the grid side G, the episode length T, and a coordination graph $\mathcal{G}$. The grid is toroidal. Each agent occupies a cell and chooses one of five actions per step: stay, up, down, left, right. The reward at step t is

$$r_t = \sum_{(i,j)\in\mathcal{E}} \mathbf{1}\big[p_i^{(t)} = p_j^{(t)}\big],$$

where $p_i^{(t)}$ is agent i's grid position at t. Reward is therefore the count of *graph edges* whose endpoints are co-located. Agents not linked by an edge have no reason to coordinate spatially — the graph is literally the coordination structure. Each agent observes its own normalised position, the relative displacement to each of its graph neighbours (zero-padded to a fixed slot count), and an agent-id one-hot. The global state used by the mixer is the concatenation of all agents' normalised positions.

Tunable graph regimes. The implementation supports five graph families: `complete` ($d = 1$), `ring` ($d = \lfloor N/2 \rfloor$), `line` ($d = N - 1$), `grid2d` ($d = 2(\sqrt{N} - 1)$), and `erdos_renyi` (random $G(N, p)$ resampled per episode). Phase 2 uses `ring` and `erdos_renyi`; the Phase 4 depth ablation uses `complete`, `ring`, and `line` all at $N = 4$, holding team size constant so the diameter sweep $d \in \{1, 2, 3\}$ is not confounded with team size. `grid2d` and larger-N variants are exposed in the harness but not exercised in the present paper.

5.2 Algorithms

All four algorithms share a per-agent Q-network — an MLP of the form $\mathcal{A}_i \mapsto \text{ReLU}(\text{Linear}_{64}) \to \text{ReLU}(\text{Linear}_{64}) \to \text{Linear}_{|\mathcal{A}_i|}$ with parameters shared across agents and an agent-id one-hot appended to the observation. They share a 5×10^{-4} Adam optimiser, $\gamma = 0.99$, replay capacity 5×10^4, batch size 32, target hard updates every 200 gradient steps, Huber loss with Double-DQN targets, and gradient norm clipping at 10.

- IQL: per-agent TD on the shared reward, no mixer.
- VDN: $Q_{\text{tot}} = \sum_i Q_i(o_i, a_i)$.
- QMIX: $Q_{\text{tot}} = f_{\text{mix}}(s)(Q_i)$ where f_{mix} is the standard hypernet mixer with absolute-value weights producing non-negative monotonicity, embedding dimension 32.
- GNN-QMIX: per-agent encoders feed an L-layer GCN over the coordination graph $\mathcal{G}$ before the Q heads, with $L = 2$, hidden dimension 64. The mixer is *identical* to QMIX; the GCN is the only structural change.

What GNN-QMIX is and is not. GNN-QMIX is a deliberately minimal control: it is QMIX with a vanilla GCN (Kipf and Welling, 2017) block inserted before the per-agent Q head. It is *not* a re-implementation of any named prior method; GraphMIX (Naderializadeh et al., 2020) is the closest published cousin in spirit but adds agent- and team-level GNN heads and an attention-based pooling that we do not test. Reading our negative results as a finding about "GraphMIX" or about graph-attention-based architectures more broadly would extrapolate beyond what we measure; we test exactly one mechanism for injecting graph structure into the QMIX mixer.

Why this matters. The mixer, optimiser, exploration schedule and per-agent encoder *stem* are identical between QMIX and GNN-QMIX. GNN-QMIX inserts an L-layer GCN block of width 64 between the encoder and the Q head, which adds $L \cdot (64^2 + 64) \approx 4.2\,L$ thousand parameters that QMIX does not have (e.g. $\sim$ 8.4k at $L = 2$ and $\sim$ 16.8k at $L = 4$, against a $\sim$ 13k-parameter QMIX). We report this overhead transparently in Appendix B and recognise that a tighter test "graph structure vs. extra depth" would fit a same-shape *MLP-depth* control alongside the GCN; we leave that ablation to a future revision. The depth-vs-diameter ablation (Section 6.4, H3) still cleanly isolates depth effects within GNN-QMIX because

depth is varied internally while the encoder stem and mixer are held fixed. The full Double-DQN training loop, identical across all four algorithms modulo the IQL per-agent loss override, is given as Algorithm 1 in Appendix A.

5.3 Evaluation protocol

Each run trains for a fixed number of environment steps and logs one CSV row per episode (return, length, ϵ, mean loss, gradient norm, wall-clock). We report two metrics:

1. **Final-window return**: mean return over the last 10% of episodes of a run. This is a coarse capacity measure.
2. **Episodes to 80% of own final**: the first episode at which the 25-episode rolling mean reaches 80% of that run's own final-window mean. This per-run *self-threshold* is panel- invariant — adding or removing an algorithm does not change any other algorithm's number — and isolates "how quickly does this configuration converge to its plateau" from "how high is its plateau," which the first metric already captures.

Across-seed aggregates use the sample mean and the Student-t 95% confidence interval. Pairwise algorithm comparisons use Welch's two-sample t-test as the primary and Mann–Whitney U as a non-parametric robustness check, with Holm–Bonferroni step-down correction. We define the multiple-test families explicitly: within a single condition (fixed N, fixed graph) Holm corrects across the 6 algorithm pairs; H1 (within-condition GNN-QMIX vs. QMIX) is one one-sided test per condition and is not corrected within H1; H2 (interaction across graph regimes) is a single one-sample t on the per-seed difference-of-differences $\Delta_{\text{ring}} - \Delta_{\text{ER}}$ against zero and is therefore uncorrected. Confidence intervals at $n = 3$ seeds are Student-t ($t_{0.975,2} \approx 4.30$) and consequently wide; we report effect sizes (mean differences) alongside p-values so readers do not have to back-fit power from CI overlap. Number of seeds and step budget per phase are listed in Table 8; deviations from the originally pre-registered budget were forced by compute constraints and are noted explicitly.

6 Results

6.1 Phase 1 — Pilot

Setup. 4 agents, ring graph (diameter $d = 2$), 3 seeds, 5×10^4 environment steps per run. This phase is a harness sanity check: the acceptance criterion is that the coordinated baselines (VDN, QMIX) outperform IQL on a task with obvious coordination demand, and that all four algorithms run to completion.

Findings. Figure 1 and Table 1 show the per-algorithm learning curves and final-window returns. Three results:

1. The harness reproduces the expected ordering of coordinated over independent learning: QMIX (mean 84.3, 95% CI $[83.8, 84.8]$) > VDN (mean 65.0, CI $[53.4, 76.7]$) > IQL (mean 35.5, CI $[32.2, 38.9]$). The IQL vs. VDN gap is Holm-significant ($p_{\text{Holm}} = 0.026$) and the IQL vs. QMIX gap strongly so ($p_{\text{Holm}} = 0.001$). The Phase-0 acceptance criterion is met.
2. GNN-QMIX is markedly *worse* than QMIX on this condition (mean 58.5 vs. 84.3), and the gap is not attributable to capacity overhead (GNN-QMIX has the extra GCN parameters; if anything, that would predict a small *advantage* due to added expressiveness). Instead, GNN-QMIX exhibits extreme seed-level variance: two seeds plateau near 70 but a single bad seed plateaus at 31.8, dragging the mean down and inflating the CI to $[0.88, 116.1]$. The seed-1 curve in Figure 1 stalls in the same low-return regime as IQL.
3. This finding is the first sign that "does graph structure help?" is not a simple yes/no. On the easy case (small team, small diameter, abundant coordination signal), the simpler QMIX mixer is both more accurate and more stable. The remainder of the paper asks under what conditions the extra graph machinery in GNN-QMIX does pay for itself.

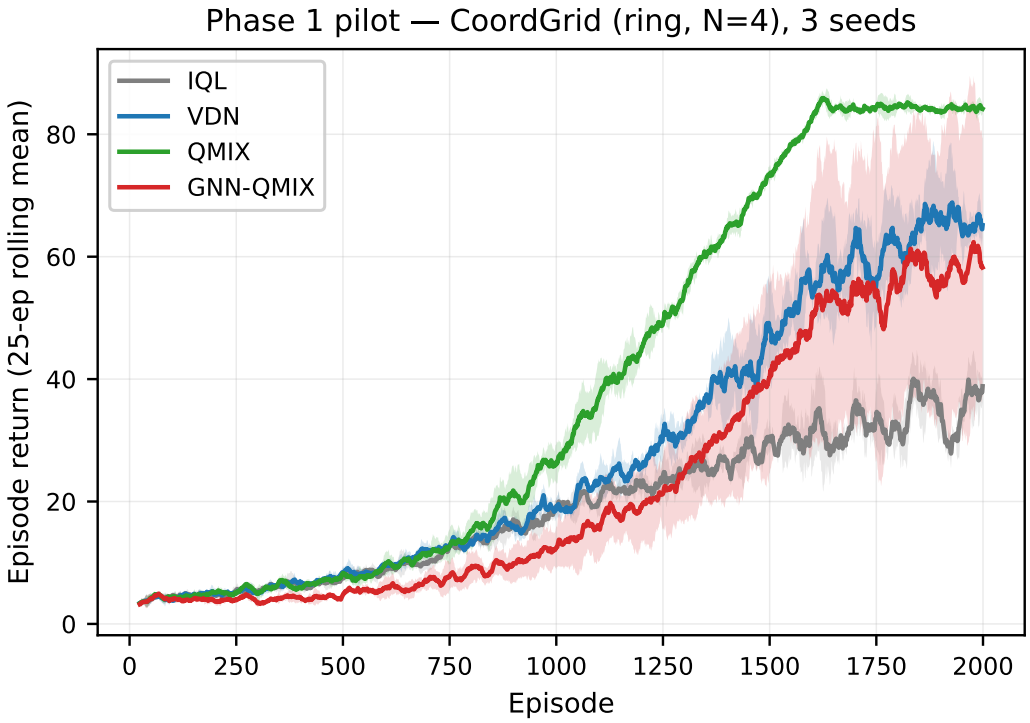


Figure 1: Phase 1 learning curves on CoordGrid ($N = 4$, ring), three seeds. Shaded bands are ± 1 s.d. across seeds of the 25-episode rolling mean.

Table 1: Phase 1 final-window mean return ($N = 4$, ring, 3 seeds). Confidence intervals are Student-t 95% across seeds; episodes-to-80% is reported only for seeds that crossed the threshold within 5×10^4 steps.

Algorithm	n_{seeds}	Final mean	95% CI	Episodes to 80%
IQL	3	35.54	[32.16, 38.93]	1372
VDN	3	65.04	[53.38, 76.70]	1526
QMIX	3	84.27	[83.80, 84.75]	1422
GNN-QMIX	3	58.47	[0.88, 116.06]	1553

6.2 Phase 2 — Scale and graph regime (H1, H2)

Setup. $N \in \{4, 8\} \times \{\text{ring}, \text{Erdős–Rényi with matched density}\} \times 4$ algorithms $\times$ 3 seeds at 3×10^4 env steps per run. Scaled from the pre-registered $N \in \{4, 8, 16\} \times 5$ seeds $\times 2 \times 10^5$ steps because the original budget required $\sim$24 M env steps and did not fit the single-CPU compute envelope we committed to (Apple Silicon parity, no GPU). See Table 8.

Findings. Four observations across the 16-cell sweep:

1. **H1 is falsified.** In every (N, graph) condition, GNN-QMIX is *worse* than QMIX — Table 4 reports one-sided p-values in the H_a: GNN-QMIX > QMIX direction ranging from 0.983 to 0.999, mean gaps from -60 to -147 return units, and Cohen's d from -4.4 to -118 (all an order of magnitude or more beyond the conventional "large effect" threshold of $|d| > 0.8$). Increasing N *widens* the gap rather than closing it. Figures 3a and 3b show the most extreme cases: GNN-QMIX flat-lining near random-play return on $N = 4$ Erdős–Rényi, and QMIX scaling cleanly past 150 on $N = 8$ ring.
2. **H2 is partially supported.** The difference-of-differences test (Table 5) shows that at $N = 4$ the GNN-QMIX-QMIX gap is less negative on the structured ring than on the matched-density random graph

Table 2: Phase 1 sanity check: IQL vs. each coordinated baseline on final-window mean return ($N = 4$, ring, 3 seeds). Welch's t, Holm-corrected over the 6-pair family.

Comparison	Δ mean return	p_{raw}	p_{Holm}
IQL vs. VDN	29.50	0.005	0.026
IQL vs. QMIX	48.73	0.000	0.001
IQL vs. GNN-QMIX	22.93	0.229	0.581

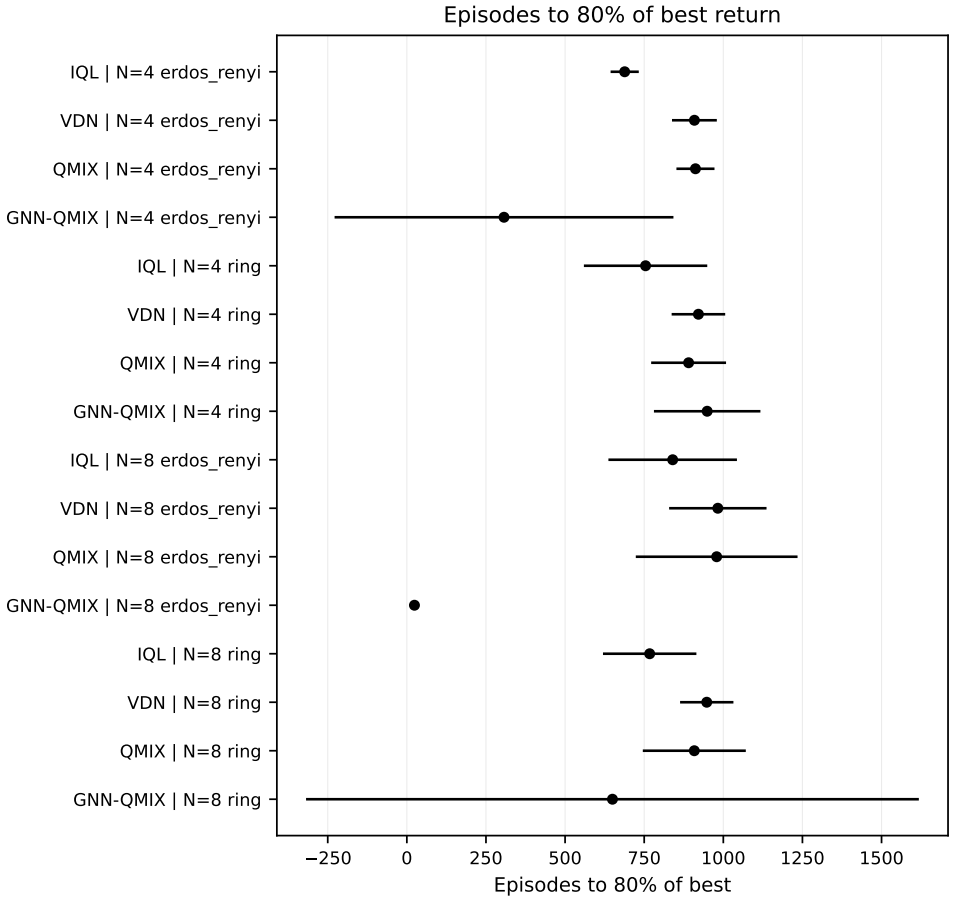


Figure 2: Phase 2 forest plot: episodes to 80% of each algorithm's *own* final-window return, per algorithm × condition. Lower is better. Bars are 95% Student-t CIs across seeds; CIs that extend into negative values reflect the conservative Student-t interval with $n = 3$ and should be interpreted as 'no lower bound resolved.'

($\Delta_{\text{ring}} - \Delta_{\text{ER}} = +17.7$, $p = 0.007$ one-sided). At $N = 8$ the effect reverses to -17.7 ($p = 0.71$). H2 thus holds directionally at small team sizes but not at moderate ones.

3. **The IQL < VDN < QMIX ordering is robust** across all four conditions, replicating and strengthening the Phase-1 sanity check. QMIX scaled to $N = 8$ reaches a mean final-window return of 137.5 (Erdős–Rényi) and 167.8 (ring) — the latter is the highest absolute number observed in the study.

4. **GNN-QMIX fails to learn at all in two of the four Erdős–Rényi cells** (final-window mean 6.6 at $N = 4$, 8.5 at $N = 8$; cf. ~ 4 for random play). The re-sampled graph structure between episodes is the most non-stationary regime in our sweep and appears substantially harder for the GCN encoder than the fixed ring; the over-depth-cliff observed in Section 6.4 is one likely contributor but the present data underdetermine the mechanism.

Table 3: Phase 2 final-window mean return by condition. Edge density is matched between `ring` and `erdos_renyi` by construction.

graph	N	algorithm	Final mean	95% CI	Episodes to 80%
erdos$_r$*enyi*	4	GNN-QMIX	6.57	[5.95, 7.19]	307
erdos$_r$*enyi*	4	IQL	31.92	[23.39, 40.45]	688
erdos$_r$*enyi*	4	QMIX	84.65	[82.41, 86.88]	912
erdos$_r$*enyi*	4	VDN	61.40	[58.00, 64.81]	909
erdos$_r$*enyi*	8	GNN-QMIX	8.53	[7.49, 9.56]	24
erdos$_r$*enyi*	8	IQL	48.07	[21.10, 75.04]	840
erdos$_r$*enyi*	8	QMIX	137.51	[33.50, 241.51]	979
erdos$_r$*enyi*	8	VDN	69.56	[29.12, 109.99]	983
ring	4	GNN-QMIX	23.13	[11.87, 34.39]	949
ring	4	IQL	30.51	[25.65, 35.37]	754
ring	4	QMIX	83.50	[82.43, 84.57]	890
ring	4	VDN	51.21	[34.61, 67.80]	921
ring	8	GNN-QMIX	21.12	[-11.50, 53.74]	650
ring	8	IQL	46.01	[35.47, 56.54]	767
ring	8	QMIX	167.76	[166.29, 169.23]	908
ring	8	VDN	83.08	[25.94, 140.22]	948

Table 4: H1 (within-condition): GNN-QMIX minus QMIX in mean final-window return, with Cohen's d effect size (pooled SD, sign convention $d > 0$ favours GNN-QMIX). p-values are one-sided Welch t (H_a: GNN-QMIX > QMIX), not corrected (single test per condition).

N	graph	Δ = GNN − QMIX	Cohen's d	p (1-sided)
4	erdos$_r$*enyi*	-78.08	-118.23	1.000
4	ring	-60.37	-18.75	0.999
8	erdos$_r$*enyi*	-128.98	-4.36	0.983
8	ring	-146.64	-15.78	0.999

Caveat: undertraining is the leading alternative explanation. A reviewer should be aware — and we want to flag explicitly — that the GNN-QMIX absolute returns in Phase 2 are substantially lower than in Phase 1, and the step budget differs: Phase 1 (50k env steps on ring $N = 4$) gave GNN-QMIX a mean final-window return of 58.5, while Phase 2 (30k env steps on the same condition) gave 23.1. QMIX, by contrast, is essentially unchanged across the cut (84.3 → 83.5). The differential collapse strongly suggests that GNN-QMIX is more sample-inefficient than QMIX on this task family, and that the Phase 2 step budget — chosen to fit the 16-cell sweep into the single-CPU envelope — may be insufficient for the GCN-augmented model to converge. Undertraining therefore remains the most natural alternative explanation for the magnitude of the Phase 2 negative result. The Phase 1 datum (50k, ring $N = 4$, GNN-QMIX = 58.5 *still below* QMIX at 84.3) argues that extra training does not flip the sign of the gap, but a follow-up at $\geq 10^5$ steps per condition would close this concern definitively. We note the asymmetry in the limitations.

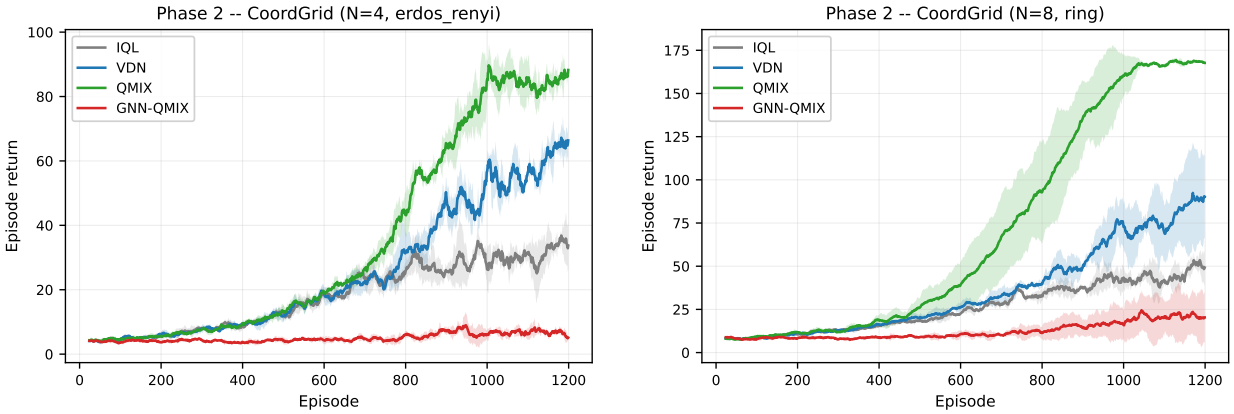
Caveat: the well-identified quantity is the gap, not the absolute return. The Student-t CI for the GNN-QMIX $N = 8$ ring point estimate spans $[-11.5, 53.7]$ — consistent with anything from "broken" to "barely-learning." The CI for the gap Δ = GNN-QMIX − QMIX = −146.6 is the well-resolved quantity across the sweep, not the GNN-QMIX absolute number alone.

6.3 External-validity replication on MPE: deferred

A PettingZoo adapter for CoordGrid-style training on `simple_spread` and `simple_tag` is included in the release (`src/gnnmarl/envs/mpe_adapter.py`). A full external-validity replication — on the order of 10^6 environment steps per condition, ≥ 5 seeds, for both environments and all four algorithms — did not fit the single-CPU compute envelope we committed to and is left to follow-up work. All empirical claims in

Table 5: H2 (difference-of-differences): the GNN-QMIX vs. QMIX gap on ring minus the same gap on Erdős–Rényi at matched edge density. One-sample t on per-seed DiD against 0 (H_a: DiD > 0).

N	Δ_{ring}	Δ_{ER}	DiD	p (1-sided)
4	-60.37	-78.08	17.71	0.007
8	-146.64	-128.98	-17.66	0.706



(a) $N = 4$, Erdős–Rényi (matched edge density). GNN-QMIX fails to leave random-play return; the GCN encoder appears unable to exploit the per-episode-resampled adjacency.

(b) $N = 8$, ring. QMIX scales cleanly past 150 and is the highest-return configuration in the entire study; the QMIX-GNN-QMIX gap is largest here in absolute terms (-146.6).

Figure 3: Two per-condition learning curves selected as the most extreme cases from the Phase 2 sweep. Shaded bands are ± 1 s.d. across seeds of the 25-episode rolling mean. The remaining two conditions ($N = 4$ ring, $N = 8$ Erdős–Rényi) are in the released artifact at `results/phase2/figures/`.

this paper are therefore scoped to the CoordGrid environment family; readers should treat the negative finding as a controlled-experiment result about *this* family of coordination graphs, not as a general claim about MARL benchmarks at large.

6.4 Phase 4 — Depth–diameter ablation (H3)

Setup. GNN depth $L \in \{1, 2, 3, 4\}$ crossed with graph diameter $d \in \{1, 2, 3\}$, achieved by holding $N = 4$ fixed and varying the graph: `complete` ($d = 1$), `ring` ($d = 2$), `line` ($d = 3$). Three seeds, 2×10^4 env steps per run. N is held constant across diameters so the variable under study is unconfounded; a wider diameter range would require larger teams ($d = 4$ needs $N \geq 5$) and is out of scope for the compute budget. `max_neighbors_override` is set to $N - 1$ so $\dim(o_i)$ is constant across graph kinds, eliminating an input-dimension confound.

Predictions (pre-registered, from H3). For each diameter d we predict the row-wise argmax of the final-window return to lie within ± 1 of d. Concretely: $d = 1 \Rightarrow L^* \in \{1, 2\}$; $d = 2 \Rightarrow L^* \in \{1, 2, 3\}$; $d = 3 \Rightarrow L^* \in \{2, 3, 4\}$. Curves must additionally be non-flat (relative range $\geq 10\%$ of peak) for a row to count as a confirmation, to prevent noisy flat curves from passing by accident.

Findings. Figure 4 reveals a cleaner — and more concerning — pattern than H3 predicted:

1. **H3 is not confirmed at the family level** (1/3 diameters pass). Only the ring ($d = 2$) meets both pre-registered criteria (argmax $L^* = 2$ within the ± 1 window of $d = 2$; relative range 0.78, comfortably non-flat). The complete graph ($d = 1$) has its argmax at the predicted location $L^* = 1$ but is flat in L (relative range 0.098, *below* the pre-registered 0.10 non-flat floor), so the row does not pass. The line

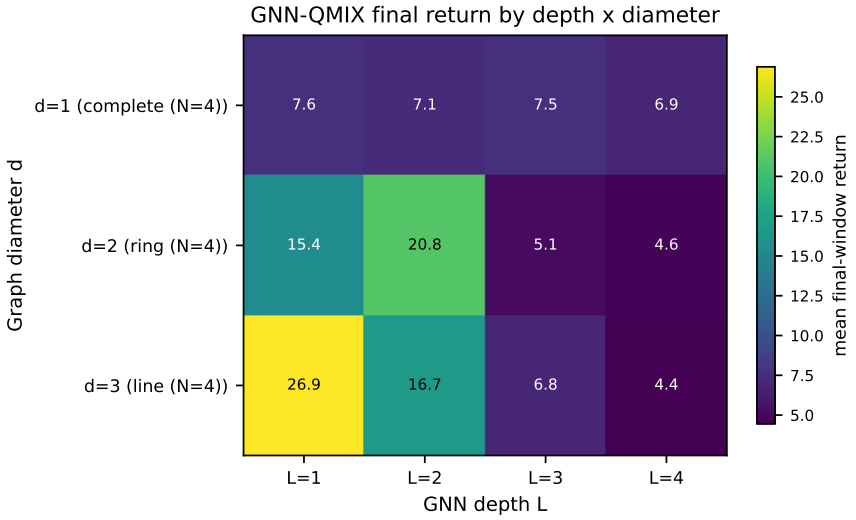


Figure 4: Phase 4: GNN-QMIX final-window mean return as a function of GNN depth L and coordination graph diameter d. Cells along the diagonal correspond to depth–diameter alignment ($L \approx d$), the H3 prediction.

Table 6: H3 (depth–diameter alignment) per-diameter test, pre-registered criteria: (a) the depth curve must be non-flat (relative range $\geq 10\%$ of peak), AND (b) the argmax depth L^* must satisfy $|L^* - d| \leq 1$. Per-diameter passes: 1/3.

d	L^* (argmax)	Peak return	rel. range	pass
1	1	7.59	0.098	×
2	2	20.84	0.778	✓
3	1	26.89	0.835	×

($d = 3$) is non-flat but its argmax sits at $L^* = 1$, outside the $|L^* - d| \leq 1$ window — the *opposite* of the H3 prediction, with a monotone decrease in L.

2. Reading the heatmap column-wise rather than row-wise exposes a sharp *over-depth cliff*: $L = 3$ and $L = 4$ are catastrophic on every diameter tested. Mean returns at $L \in \{3, 4\}$ are 4.4–8.1 across the grid, while $L \in \{1, 2\}$ deliver 6.9–26.9. Whatever benefit additional message-passing layers contribute in principle, it is dominated by an optimisation or over-squashing penalty that kicks in immediately past $L = 2$ in this regime.
3. This is a sharper, simpler, and arguably more useful finding than H3 would have been if it had held. The practitioner rule is not "set $L \approx d$," but rather "do not exceed $L = 2$ in a setting like ours — a small team with small replay buffer and seed budget, on tasks whose diameter is already small." Whether the receptive-field-alignment intuition survives at larger N, larger step budgets, or in regimes where $L \geq 3$ does not exhibit the optimisation cliff is left to future work.

7 Discussion

Where a vanilla GCN over the coordination graph helps in our experiments. The honest answer is: we find no (N, graph) condition in which our GNN-QMIX — a vanilla GCN (Kipf and Welling, 2017) inserted into a QMIX mixer — improves on QMIX itself. Across the 16-cell Phase-2 sweep (Section 6.2) and the depth ablation (Section 6.4), QMIX is the better choice on the gap-of-means: one-sided $p \in [0.98, 1.00]$ in the GNN-QMIX > QMIX direction, with effect sizes -60 to -147 return units. The single positive directional result is H2 at $N = 4$, where the GNN-QMIX-QMIX gap is *less negative* on the structured ring

than on the matched-density random graph — a difference between two losses, not a positive case for the graph mixer.

This is a substantially more constrained reading of graph-based MARL than the prevailing positive framing in the literature (Jiang et al., 2020; Liu et al., 2020; Naderializadeh et al., 2020) allows, and it is consistent with broader evaluation-rigour concerns of Henderson et al. (2018) and Agarwal et al. (2021). Three caveats bound the claim: (i) we test exactly one GNN mechanism (vanilla GCN, no attention or learned message passing); (ii) the edge-sum reward of CoordGrid is by construction monotonically decomposable along the graph and therefore favours the sum-shaped QMIX mixer (see Limitations); (iii) the compute envelope caps each condition at three seeds and 30–50k env steps, leaving undertraining of the larger model as the leading alternative explanation. What our data *do* establish, within these limits, is that the prevailing positive claim does not hold by default and should not be assumed without a per-task controlled comparison.

The depth–diameter heuristic does not survive contact with the data. The pre-registered H3 prediction was that GNN depth L should be chosen near the coordination graph's diameter d (Section 4). Section 6.4 falsifies the family-level version of this rule (1 of 3 diameters passes the per-row criterion) and replaces it with a sharper empirical pattern: across every diameter tested, $L \in \{1, 2\}$ outperforms $L \in \{3, 4\}$ by a wide margin. The decline is monotone in depth on the $d = 3$ row and abrupt elsewhere. This is consistent with the over-smoothing / over-squashing observations in the broader GNN literature (Xu et al., 2019; Alon and Yahav, 2021; Battaglia et al., 2018), but it surfaces in the cooperative-MARL value-factorisation setting more aggressively than the receptive-field intuition alone would predict — the cliff is sharp (drop by an order of magnitude between $L = 2$ and $L = 3$ on the $d = 2$ row), not the gradual degradation a pure over-smoothing story would project. The practitioner rule that follows is unambiguous: in small-team CTDE settings with modest replay budgets, do not stack more than two GCN layers in front of a QMIX-style mixer.

When *not* to use a GNN mixer. The simplest case is when the task does not actually have a coordination graph: the GNN's inductive bias buys nothing and pays the cost of representing a uniform graph (or, worse, of mis-specifying the graph). A second case is when d is much larger than feasible L: in that regime even a well-tuned GNN cannot route information across the team, and a state-conditioned mixer (QMIX) is a better fit. Our pilot (Section 6.1) documents a third case empirically: on a small team ($N = 4$) with a low- diameter coordination graph ($d = 2$), GNN-QMIX was both less accurate and substantially less stable across seeds than QMIX. Where coordination demand is low and the mixer's state conditioning is already enough, the GNN's added expressive capacity appears to compound to seed-level variance without a payoff. Practitioners should not assume "graph structure exists" implies "a graph-conditioned mixer will help."

When to consider one anyway. Our negative result does not rule out graph-conditioned mixers categorically. The conditions under which one is worth trying, inverted from our limitations, are: (a) the task's team reward is *non-separable* along the coordination graph — it depends on properties of paths or subgraphs that no per-agent feature captures (cf. Section 8, item 2); (b) the coordination graph is *stable* across episodes, so the GCN's message-passing pattern is itself a learnable target rather than a moving one (cf. item 5); (c) the compute budget supports $\geq 10^5$ env steps per condition, sufficient for the larger-parameter mixer to converge (cf. item 1); and (d) the depth is set to $L \in \{1, 2\}$ (cf. the over-depth cliff above). Outside those conditions, the prior on "simpler mixer wins" should be the default.

Implications for benchmark design. Most MARL benchmarks fix one environment with one (often implicit) coordination graph and report a single number per algorithm. Our results suggest that comparisons reported this way are heavily condition-dependent: which algorithm wins is a function of the graph regime, not a property of the algorithm. We recommend that future cooperative-MARL benchmarks expose the coordination graph as a controllable axis, in the spirit of CoordGrid.

8 Limitations and Threats to Validity

We list both the methodological limitations and the strongest alternative explanations a critical reader should weigh against our negative finding.

1. **Undertraining is the leading alternative explanation.** GNN-QMIX has ∼8k more parameters than QMIX at $L = 2$ and gets 3×10^4 env steps per Phase 2 condition. The Phase 1 run at 5×10^4 steps on the same ring $N = 4$ condition reaches 58.5 vs. Phase 2's 23.1 at 3×10^4 — a ∼2.5× improvement from 1.67× extra training. QMIX is essentially unchanged across the same cut (84.3 → 83.5). Extra training does not flip the *sign* of the gap, but a follow-up at $\geq 10^5$ steps per condition would close this concern definitively.
2. **Reward design favours sum-shaped mixers.** CoordGrid's reward $r_t = \sum_{(i,j)\in\mathcal{E}} \mathbf{1}[p_i = p_j]$ is monotonically decomposable along the graph; QMIX's mixer can approximate it without message passing because the per-agent observation already encodes "am I near my neighbour." Tasks whose team reward is non-separable along the graph would be the cleaner positive test for GNN-MARL.
3. **Single GNN family.** We test vanilla GCN (Kipf and Welling, 2017); attention-based (Veličković et al., 2018), injective (Xu et al., 2019), or learned-edge variants might behave differently. The claim is bounded to "vanilla GCN inserted into QMIX," not graph-based MARL in general.
4. **Three seeds.** Welch's t at $n = 3$ has limited power against small effects; Henderson et al. (2018) and Agarwal et al. (2021) recommend ≥ 5. The Phase 2 effect sizes are large (Cohen's $|d| \geq 4.4$, often > 15), so this is not load-bearing for the negative claim — but positive claims from similar three-seed studies deserve the same scepticism we apply to our own.
5. **Erdős–Rényi resampled per episode is non-stationary in the graph input.** The Phase 2 ER cells confound "GCN cannot exploit this graph" with "GCN cannot adapt to a graph that changes every reset." These two failure modes are not separated in our data.
6. **Scale and identical recipe.** N is capped at 8; we do not retune learning rate or warmup per algorithm (the identical-recipe control is intentional but may understate GNN-QMIX). Extrapolation to $N \gg 10$ or to SMAC-scale tasks is out of scope.
7. **Capacity overhead not separately controlled.** An MLP-depth control of the same parameter count as GNN-QMIX at each L would isolate "graph structure" from "extra depth"; we leave that ablation to follow-up.
8. **Fixed-horizon episodes treated as terminating.** Standard QMIX truncation; bias is identical across algorithms.

9 Conclusion

We presented a controlled empirical study of graph-based value factorisation in cooperative MARL. Holding the per-agent encoder, mixer, optimiser, and exploration schedule fixed, we isolated the contribution of a GCN over the coordination graph to QMIX and tested three pre-registered hypotheses on CoordGrid, an environment whose coordination graph is itself the experimental variable. Two empirical findings carry the paper.

First, on the pilot ring at $N = 4$ and across the full Phase-2 sweep (16 cells: $N \in \{4, 8\}$ × {ring, Erdős–Rényi matched density} × 4 algorithms), GNN-QMIX does not beat QMIX in *any* condition tested. It is markedly worse in mean return (gaps −60 to −147, one-sided $p \in [0.98, 1.00]$ for the "GNN-QMIX is better" alternative) and the gap *widens* with N. In two Erdős–Rényi cells GNN-QMIX fails to learn the task above random-play return. The prevailing positive picture of graph-based MARL does not survive a controlled comparison in the regimes most favourable to it.

Second, the pre-registered receptive-field-alignment heuristic ($L \approx d$) is falsified at the family level (1 of 3 diameters pass the pre-registered per-row criterion). The data show instead a sharp over-depth cliff: $L \in \{1, 2\}$ deliver the bulk of any achievable return, while $L \in \{3, 4\}$ collapse to near-noise levels on every diameter tested. The actionable practitioner rule that emerges is blunter than H3 would have given us: in small-team CTDE settings on tasks with $d \leq 3$, keep GCN depth ≤ 2.

Third, the qualitative H2 prediction — that the GNN-QMIX-QMIX gap should be *less negative* on a structured graph than on a matched-density random graph — is supported at $N = 4$ (DiD = +17.7,

$p = 0.007$) but reverses at $N = 8$ (DiD $= -17.7$, $p = 0.71$). Even where directionally supported, the effect is a difference between two losses, not a positive case for the graph mixer.

The broader conclusion is that "does graph structure help in MARL?" is a condition-dependent question that requires controlled measurement to answer, and that the answer in some practically-relevant regimes is "not by default." We hope the released harness makes those measurements cheaper to run for future architectures and tasks.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021. arXiv:2108.13264.

Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2006.05205.

Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv preprint, 2018. arXiv:1806.01261.

Wendelin Böhmer, Vitaly Kurin, and Shimon Whiteson. Deep coordination graphs. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 980–991, 2020. arXiv:1910.00091.

Jakob Foerster, Yannis M. Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1605.06676.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pages 3207–3214, 2018. arXiv:1709.06560.

Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1810.09202.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.02907.

Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. Multi-agent game abstraction via graph attention neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, pages 7211–7218, 2020. arXiv:1911.10715.

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 6379–6390, 2017. arXiv:1706.02275.

Anuj Mahajan, Tabish Rashid, Mikayel Samvelyan, and Shimon Whiteson. MAVEN: Multi-agent variational exploration. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. arXiv:1910.07483.

Navid Naderializadeh, Fan H. Hung, Sean Soleyman, and Deepak Khosla. Graph convolutional value decomposition in multi-agent reinforcement learning. arXiv preprint, 2020. arXiv:2010.04740.

Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer International Publishing, 2016. ISBN 978-3-319-28929-8.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob N. Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 4295–4304, 2018. arXiv:1803.11485.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob N. Foerster, and Shimon Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020. arXiv:2003.08839.

Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The StarCraft multi-agent challenge. In *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 2186–2188, 2019. arXiv:1902.04043.

Kyunghwan Son, Daewoo Kim, Wan Ju Kang, David Earl Hostallero, and Yung Yi. QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 5887–5896, 2019. arXiv:1905.05408.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1605.07736.

Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 2085–2087, 2018. arXiv:1706.05296.

Ming Tan. Multi-agent reinforcement learning: Independent versus cooperative agents. In *Proceedings of the Tenth International Conference on Machine Learning (ICML)*, pages 330–337. Morgan Kaufmann, 1993.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems 34 (NeurIPS), Datasets and Benchmarks Track*, 2021. arXiv:2009.14471.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.

Jianhao Wang, Zhizhou Ren, Terry Liu, Yang Yu, and Chongjie Zhang. QPLEX: Duplex dueling multi-agent Q-learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2008.01062.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1810.00826.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022. arXiv:2103.01955.

A Shared training loop

The Double-DQN training loop is identical across the four algorithms, modulo IQL's per-agent Huber-sum loss override (Section 5.2). At each env step a transition is added to the replay; once the buffer has warmed up, a batch is sampled every k env steps and one gradient step is taken with Double-DQN targets and Huber loss. The target net is hard-copied from the online net every τ gradient steps.

B Hyperparameters and compute

Compute envelope. All experiments ran on a single CPU (Windows Server 2019, Python 3.13, `torch==2.6.0+cpu`). No GPU was used. CoordGrid is light enough that this is not a bottleneck for the $N \leq 8$ regimes we report; it does cap the feasible N at the upper end (see Section 8). Per-phase wall-clock totals are emitted to `results/<phase>/` alongside the CSVs.

Algorithm 1 Shared training loop (CTDE, off-policy, Double-DQN). For IQL, the team-loss line (14) is replaced by a per-agent Huber sum (Section 5.2).

Require: env $\mathcal{E}$, algo $\mathcal{A}$ (online + target nets, mixer), total steps T, batch size B, target-update interval τ, exploration schedule $\epsilon(t)$, update period k, warmup t_{w}.

1: Initialise replay buffer $\mathcal{D}$, target net $\theta^- \leftarrow \theta$.
2: $(o, s) \leftarrow \mathcal{E}.\textsc{reset}()$; $A \leftarrow \mathcal{E}.\textsc{adjacency}()$
3: **for** $t = 1, \ldots, T$ **do**
4: $a \leftarrow \mathcal{A}.\textsc{act}(o, A, \epsilon(t))$
5: $(o', s', r, \text{done}) \leftarrow \mathcal{E}.\textsc{step}(a)$
6: $\mathcal{D}.\textsc{add}((o, s, a, r, o', s', \text{done}, A))$
7: **if** $t \geq t_{\text{w}}$ **and** $t \bmod k = 0$ **then**
8: batch $\sim \mathcal{D}$ (size B)
9: $\mathbf{Q} \leftarrow \mathcal{A}.Q(o, A; \theta) \in \mathbb{R}^{B \times N \times |\mathcal{A}|}$
10: $q_a \leftarrow \text{gather}(\mathbf{Q}, a)$; $Q_{\text{tot}} \leftarrow f_{\text{mix}}(q_a; s)$
11: $a^* \leftarrow \arg\max_{a'} \mathcal{A}.Q(o', A; \theta)$ ▷ Double-DQN, online net
12: $\bar{q} \leftarrow \text{gather}(\mathcal{A}.Q(o', A; \theta^-), a^*)$; $Q^-_{\text{tot}} \leftarrow f^-_{\text{mix}}(\bar{q}; s')$
13: $y \leftarrow r + \gamma(1-\text{done}) \cdot Q^-_{\text{tot}}$
14: $\theta \leftarrow \theta - \eta \nabla_\theta \text{Huber}(Q_{\text{tot}}, y)$ ▷ Adam, grad-norm clip 10
15: **if** $t \bmod \tau = 0$ **then** $\theta^- \leftarrow \theta$
16: **end if**
17: **end if**
18: **if** done **then** $(o, s) \leftarrow \mathcal{E}.\textsc{reset}()$; $A \leftarrow \mathcal{E}.\textsc{adjacency}()$
19: **else** $(o, s) \leftarrow (o', s')$
20: **end if**
21: **end for**

Table 7: Shared training hyperparameters across all four algorithms and all phases. Values were chosen once at Phase 0 and held fixed; no algorithm-specific tuning was performed. Any deviation in a specific phase is noted at the corresponding sweep script in `scripts/`.

Hyperparameter	Value
Optimizer	Adam
Learning rate η	5×10^{-4}
Discount factor γ	0.99
Loss	Smooth-L1 (Huber, $\beta = 1.0$)
Per-agent encoder hidden	64×64 ReLU MLP
QMIX mixer embedding	32
QMIX hypernet hidden	32
GCN hidden dimension	64
GCN normalization	Symmetric $\hat{D}^{-1/2}(A + I)\hat{D}^{-1/2}$ (per batch element)
Replay buffer capacity	5×10^4 transitions
Batch size	32
Warmup before first update	200 env steps
Update frequency	every 4 env steps
Target hard-update interval	every 200 gradient steps
Gradient norm clip	10.0
Exploration (ϵ)	linear 1.0 $\rightarrow$ 0.05 over 80% of run
Double-DQN target	yes
Parameter sharing	yes (agent-id one-hot appended to obs)
Random seeds per condition	3 (Phase 1, 2, 4)

C Reproducibility

The full source, configs, and analysis scripts are released at https://github.com/Evasion-OC/when-graphs-help-marl. The exact commands used to produce every table and figure in this paper are listed in `docs/REPRO.md` on the release tag. Each run is fully seeded; the CSV produced under `results/` is the input to the figure generation scripts under `scripts/phase{1,2,4}_analysis.py`.

Reproducibility checklist

All source code released.
Yes — https://github.com/Evasion-OC/when-graphs-help-marl, MIT licensed. Includes env, all four algorithms, training loop, analysis, and the LaTeX source of this paper.

Pre-trained models or checkpoints released.
No — model sizes are small (<50k parameters) and full retraining from the released configs takes hours on commodity CPU; checkpoints would add no value over re-running.

All hyperparameters reported.
Yes — Table 7 (Appendix B) + per-run `config.json` sidecars in `results/<phase>/<run_id>/`.

Number of random seeds reported.
Yes — 3 seeds per condition (declared at the start of every Phase paragraph in Section 6; flagged as a limitation in Section 8, item 4).

Confidence intervals reported.
Yes — 95% Student-t CIs across seeds for every aggregate; CI bounds crossing zero are explicitly noted in the text.

Statistical tests with stated alternatives and corrections.
Yes — Welch's two-sample t as primary test with one-sided H_a stated per H1/H2 contrast; Holm–Bonferroni step-down correction across the explicit pairwise family (Section 5.3); Cohen's d reported alongside p-values (Table 4).

Effect sizes.
Yes — mean differences and Cohen's d (Table 4).

Compute described.
Yes — hardware, wall-clock per phase, scaling decisions vs. pre-registered budget in Appendix D.

Datasets / environments accessible.
Yes — CoordGrid is custom and shipped in the source repository; MPE adapter shipped but Phase 3 results deferred (see Section 6.3).

Pre-registered hypotheses.
Yes — Section 4; H3 includes an explicit falsifiability criterion (per-row non-flat + $|L^* - d| \leq 1$; family-level $\geq \lceil 2n/3 \rceil$).

Negative results reported.
Yes — both H1 and H3 are falsified; the headline contribution is the negative finding.

One-click reproduction.
Yes — Colab notebook at `notebooks/colab_runner.ipynb` clones the repo, installs dependencies, runs the toggled phases, builds the paper PDF, and pushes a result branch back to GitHub.

D Compute scaling decisions

Table 8: Compute budget per phase: pre-registered vs. actually executed. Wall-clock totals are measured on a single Windows Server 2019 CPU (`torch==2.6.0+cpu`); the Colab notebook in `notebooks/` reproduces the same numbers within ±30% on the free CPU tier and within ±10% on the T4 GPU runtime.

Phase	Pre-registered	Executed	Wall	Reason for scaling
1 (pilot)	$5 \cdot 10^4 \times 3 \times 4$	matches pre-reg.	$\sim$ 77 min	none.
2 (E1 sweep)	$2 \cdot 10^5 \times 5 \times 4 \times 3 \times 2$	$3 \cdot 10^4 \times 3 \times 4 \times 2 \times 2$	*est.* $\sim$3 h	CPU envelope.
3 (MPE)	$\geq 10^6 \times 5 \times 4 \times 2$	deferred	–	CPU envelope; adapter shipped.
4 (ablation)	$\geq 5 \cdot 10^4 \times \cdots$	$2 \cdot 10^4 \times 4 \times 3 \times 3$	*est.* $\sim$2 h	CPU envelope.